

EasyTaxi Palma

Aplicación web para la reserva y petición de taxis en Palma de
Mallorca

Sebastià Romaguera Camps

Proyecto Intermodular
Ciclo Formativo de Grado Superior
Desarrollo de Aplicaciones Web (IFC33X)

Marzo 2026

Versión: UD1B - Sistema de viajes implementado

Este documento presenta el desarrollo del proyecto EasyTaxi Palma, una aplicación web moderna para la gestión y reserva de taxis en Palma de Mallorca, desarrollada como parte del Proyecto Intermodular del ciclo DAW.

Índice

Resumen	4
1 Introducción y contexto personal	5
1.1 Problemática actual del sector	5
1.2 Motivación del proyecto	5
2 Objetivos del proyecto	7
2.1 Objetivo general	7
2.2 Objetivos específicos	7
3 Tecnologías utilizadas	9
3.1 Backend	9
3.1.1 Node.js	9
3.1.2 Express.js	9
3.1.3 MongoDB + Mongoose	9
3.1.4 Librerías de seguridad	10
3.2 Frontend	10
3.2.1 HTML5, CSS3 y JavaScript ES6+	10
3.2.2 Phosphor Icons	10
3.2.3 Fetch API	10
3.3 Herramientas de desarrollo	10
3.4 Tecnologías futuras planificadas	11
4 Arquitectura y diseño del sistema	12
4.1 Arquitectura general	12
4.2 Estructura de directorios	12
4.3 Modelo de datos	13
4.4 API REST - Endpoints	15
4.5 Flujo de autenticación	15
5 Desarrollo e implementación	16
5.1 Estado actual del proyecto (Versión UD1B)	16
5.1.1 Sistema de autenticación completo	16
5.1.2 Interfaz de usuario	16
5.1.3 Nuevas páginas del frontend (UD1B)	17
5.1.4 Sistema de viajes (UD1B)	18
5.1.5 Despliegue en Vercel	18
5.1.6 Características de diseño	19
5.2 Backend implementado	19

5.2.1	Servidor Express	19
5.2.2	Controlador de autenticación	20
5.3	Base de datos	20
5.4	Sistema de ubicaciones	21
6	Impacto y aspectos éticos	22
6.1	Accesibilidad	22
6.2	Usabilidad	22
6.3	Protección de datos (RGPD)	23
6.3.1	Medidas técnicas implementadas	23
6.3.2	Medidas organizativas planificadas	23
6.4	Licencias de software	24
6.5	Diversidad e inclusión	24
6.6	Impacto medioambiental	24
7	Metodología de desarrollo	25
7.1	Metodología ágil con Trello	25
7.2	Control de versiones con Git	25
7.3	Testing y validación	25
8	Próximos pasos y trabajo futuro	26
8.1	Fase UD1B - Sistema de viajes (Completada)	26
8.2	Fase UD1C - Geolocalización y mapas	26
8.3	Fase UD2 - Sistema de reservas	26
8.4	Fase UD3 - Funcionalidades avanzadas	27
8.5	Mejoras de seguridad	28
8.6	Optimizaciones	28
8.7	Expansión futura	28
9	Conclusiones	29
9.1	Logros alcanzados	29
9.2	Competencias adquiridas	29
9.3	Retos superados	30
9.4	Reflexión personal	30
9.5	Viabilidad del proyecto	31
10	Bibliografía y referencias	32
10.1	Documentación oficial	32
10.2	Libros y recursos educativos	32
10.3	Artículos y guías web	32
10.4	Herramientas y librerías	33

10.5 Legislación y normativa	33
10.6 Recursos de consulta durante el desarrollo	33
Anexos	34
Anexo A: Capturas de pantalla	34
Anexo B: Diagrama de base de datos	34
Anexo C: Flujos de usuario	34
Anexo D: Código fuente relevante	35
Declaración de autoría	36

Resumen

EasyTaxi Palma es una aplicación web diseñada para modernizar el sector del taxi en Palma de Mallorca mediante la digitalización del proceso de reserva y solicitud de taxis. El proyecto nace de la experiencia personal del autor trabajando como taxista, identificando las carencias tecnológicas del sector frente a plataformas modernas como Uber o Cabify.

La aplicación implementa un sistema completo de registro y autenticación de usuarios, una interfaz intuitiva y responsive, y una arquitectura cliente-servidor robusta utilizando tecnologías modernas como Node.js, Express, MongoDB y JavaScript vanilla. El sistema garantiza la seguridad mediante encriptación de contraseñas con bcrypt y validación exhaustiva de datos tanto en frontend como en backend.

El estado actual del proyecto (versión UD1B) incluye un sistema funcional de gestión de usuarios con registro, login y perfil personalizado, así como un completo sistema de solicitud y seguimiento de viajes en tiempo real. La versión incorpora los modelos de datos **Conductor** y **Viaje**, una API REST ampliada con endpoints de viajes, asignación automática del conductor disponible más próximo, seguimiento del progreso mediante un timeline animado, cálculo estimado de precios y tiempos, sistema de valoraciones y posibilidad de cancelación. El proyecto está configurado para despliegue en Vercel como función serverless. Las próximas fases incorporarán autenticación JWT y geolocalización en tiempo real con mapas interactivos Leaflet.

Palabras clave: Aplicación web, taxi, Node.js, MongoDB, Express, JavaScript, geolocalización, sistema de reservas, API REST.

1. Introducción y contexto personal

He elegido este proyecto porque el verano pasado estuve trabajando como taxista en Palma de Mallorca durante cinco meses y pude ver de primera mano los problemas que tiene el sector. Muchos procesos siguen siendo muy tradicionales y eso hace que el servicio sea menos ágil y cómodo para los clientes.

Con la llegada de las VTC (Vehículos de Transporte con Conductor), como Uber o Cabify, el taxi tradicional se ha quedado atrás en cuanto a tecnología, rapidez y facilidad de uso. Por eso considero necesario que el sector se modernice y ofrezca una alternativa digital que permita competir con esas empresas.

Mi idea es crear una aplicación web que permita reservar y solicitar taxis en Palma de forma sencilla, segura, rápida y con una geolocalización a tiempo real. Con los conocimientos que adquirí el año pasado en el ciclo de Desarrollo de Aplicaciones Web y todo lo que voy a aprender este año, creo que puedo aportar una solución real a este problema y demostrar mi progreso como desarrollador.

1.1. Problemática actual del sector

Durante mi experiencia como taxista, identifiqué varios problemas fundamentales:

- **Falta de digitalización:** La mayoría de solicitudes se realizan mediante llamadas telefónicas a centralitas, un proceso lento y poco eficiente.
- **Kilómetros en vacío:** Los taxistas recorren grandes distancias sin pasajeros buscando clientes, lo que incrementa costes y reduce rentabilidad.
- **Desventaja competitiva:** Plataformas como Uber ofrecen una experiencia de usuario superior con seguimiento en tiempo real, estimación de precios y pago digital.
- **Accesibilidad limitada:** Personas mayores o con movilidad reducida tienen dificultades para solicitar taxis de forma tradicional.
- **Falta de transparencia:** Los usuarios no tienen visibilidad del tiempo de espera ni de la ubicación del taxi asignado.

1.2. Motivación del proyecto

Este proyecto representa para mí la oportunidad de:

1. Aportar una solución práctica a un problema real que conozco de primera mano.

-
2. Aplicar los conocimientos adquiridos en el ciclo formativo en un contexto profesional.
 3. Desarrollar un proyecto completo desde cero, incluyendo análisis, diseño, implementación y documentación.
 4. Demostrar mi evolución como desarrollador web full-stack.
 5. Crear una base sólida que pueda evolucionar hacia un producto real en el futuro.

2. Objetivos del proyecto

2.1. Objetivo general

Desarrollar una aplicación web funcional que permita solicitar, reservar y gestionar taxis en Palma de Mallorca con geolocalización a tiempo real de manera rápida, segura y accesible.

2.2. Objetivos específicos

1. **Implementar un sistema de registro y autenticación de usuarios y conductores.**
 - Registro con validación de datos (nombre, email, contraseña, teléfono).
 - Login con autenticación segura.
 - Gestión de sesiones de usuario.
 - Página de perfil personalizada.
2. **Diseñar un mapa interactivo que muestre los taxis disponibles y permita hacer reservas en tiempo real.**
 - Integración de Leaflet para visualización de mapas.
 - Geolocalización del usuario.
 - Visualización de taxis cercanos en el mapa.
 - Cálculo de rutas óptimas.
3. **Crear un sistema de gestión de reservas con confirmación y estado del servicio.**
 - Estados: pendiente, confirmado, en camino, completado, cancelado.
 - Notificaciones en tiempo real del estado del servicio.
 - Historial de reservas del usuario.
 - Sistema de valoraciones y comentarios.
4. **Garantizar la seguridad y protección de datos personales.**
 - Encriptación de contraseñas con bcrypt.
 - Implementación de tokens JWT para autenticación.
 - Validación exhaustiva de datos en cliente y servidor.
 - Cumplimiento de normativas de protección de datos.

5. Diseñar una interfaz accesible e intuitiva.

- Diseño responsive adaptable a móviles, tablets y escritorio.
- Interfaz multiidioma (español, inglés, catalán).
- Cumplimiento de estándares de accesibilidad web.
- Sistema de mensajes modales personalizados.

3. Tecnologías utilizadas

La elección del stack tecnológico se ha realizado considerando criterios de eficiencia, escalabilidad, facilidad de desarrollo y experiencia personal adquirida durante la formación y prácticas profesionales.

3.1. Backend

3.1.1. Node.js

Node.js es el entorno de ejecución principal del backend. He elegido Node.js por varias razones:

- Es el lenguaje que he aprendido y utilizado en la empresa durante mis prácticas.
- Permite usar JavaScript tanto en frontend como en backend, manteniendo consistencia.
- Gran ecosistema de paquetes mediante npm.
- Excelente rendimiento para aplicaciones en tiempo real.
- Amplia comunidad y documentación.

3.1.2. Express.js

Framework web minimalista para Node.js que facilita la creación de APIs REST. Proporciona:

- Sistema de routing robusto y flexible.
- Middleware para procesamiento de peticiones.
- Manejo simplificado de HTTP requests y responses.
- Fácil integración con otros módulos.

3.1.3. MongoDB + Mongoose

MongoDB es la base de datos NoSQL elegida por:

- Flexibilidad de esquema, ideal para desarrollo ágil.
- Excelente rendimiento con grandes volúmenes de datos.
- Fácil escalabilidad horizontal.

- **Licencia open source** (Community Edition).
- **Mongoose** proporciona ODM (Object Document Mapping) con validación y tipado.

3.1.4. Librerías de seguridad

- **bcryptjs**: Encriptación de contraseñas mediante algoritmo de hashing robusto.
- **cors**: Gestión de Cross-Origin Resource Sharing para comunicación segura cliente-servidor.
- **dotenv**: Gestión de variables de entorno para configuraciones sensibles.

3.2. Frontend

3.2.1. HTML5, CSS3 y JavaScript ES6+

El frontend está desarrollado con tecnologías web estándar:

- **HTML5**: Estructura semántica y accesible.
- **CSS3**: Diseño responsive con flexbox, grid y media queries.
- **JavaScript ES6+**: Lógica de aplicación con características modernas (arrow functions, async/await, destructuring).

3.2.2. Phosphor Icons

Librería de iconos moderna y completa que proporciona más de 6,000 iconos consistentes y personalizables.

3.2.3. Fetch API

API nativa de JavaScript para realizar peticiones HTTP asíncronas al backend, reemplazando librerías externas como Axios.

3.3. Herramientas de desarrollo

- **Git/GitHub**: Control de versiones y repositorio remoto.
- **VS Code**: Editor de código con extensiones para desarrollo web.
- **Nodemon**: Recarga automática del servidor durante el desarrollo.
- **Docker**: Contenedorización de MongoDB para facilitar el despliegue.
- **Trello**: Gestión de tareas y metodología ágil.

3.4. Tecnologías futuras planificadas

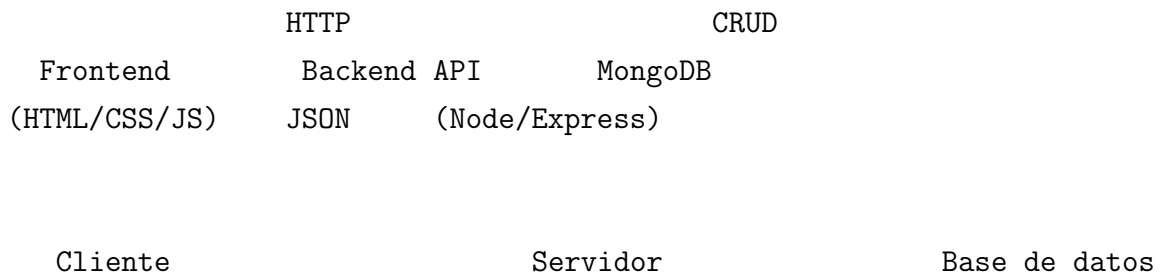
- **JSON Web Tokens (JWT):** Autenticación stateless y segura.
- **Leaflet:** Librería de mapas interactivos open source.
- **OpenStreetMap:** Proveedor de datos cartográficos gratuito.
- **Socket.io:** Comunicación en tiempo real entre cliente y servidor.

4. Arquitectura y diseño del sistema

4.1. Arquitectura general

EasyTaxi Palma sigue una arquitectura **cliente-servidor de tres capas**:

1. **Capa de presentación (Frontend):** Interfaz web SPA (Single Page Application).
2. **Capa de lógica de negocio (Backend):** API REST con Express.
3. **Capa de datos (Base de datos):** MongoDB con Mongoose ODM.



4.2. Estructura de directorios

La estructura del proyecto está organizada de forma modular:

```

1 Proyecto-Intermodular/
2   backend/
3     node_modules/
4     src/
5       config/
6         database.js
7       controllers/
8         authController.js
9         viajeController.js
10      models/
11        Usuario.js
12        Conductor.js
13        Viaje.js
14      routes/
15        authRoutes.js
16        viajeRoutes.js
17      .env
18      seed-conductores.js
19      package.json
20      server.js

```

```
21     frontend/  
22         assets/  
23             css/  
24                 styles.css  
25             images/  
26                 EasyTaxiImagen.png  
27             js/  
28                 auth.js  
29             index.html  
30             login.html  
31             register.html  
32             perfil.html  
33             conductores.html  
34             ayuda.html  
35             viaje-en-curso.html  
36             historial-viajes.html  
37     docs/  
38         tecnica.md  
39         tfg_easytaxi.tex  
40     docker-compose.yml  
41     vercel.json  
42     README.md
```

Listing 1: Estructura del proyecto

4.3. Modelo de datos

El modelo de datos incluye tres entidades principales: Usuario, Conductor y Viaje.

```
1 const usuarioSchema = new mongoose.Schema({  
2     nombre: {  
3         type: String,  
4         required: [true, 'El nombre es obligatorio'],  
5         trim: true  
6     },  
7     email: {  
8         type: String,  
9         required: [true, 'El email es obligatorio'],  
10        unique: true,  
11        lowercase: true,  
12        trim: true  
13    },  
14    password: {  
15        type: String,  
16        required: [true, 'La contraseña es obligatoria'],  
17        minlength: 6  
18    },
```

```
19     telefono: {
20         type: String,
21         required: [true, 'El telefono es obligatorio']
22     },
23     fechaRegistro: {
24         type: Date,
25         default: Date.now
26     }
27 });
```

Listing 2: Modelo Usuario (Mongoose)

```
1 const conductorSchema = new mongoose.Schema({
2     nombre:      { type: String, required: true, trim: true },
3     apellidos:   { type: String, required: true, trim: true },
4     telefono:    { type: String, required: true, trim: true },
5     vehiculo: {
6         marca:      { type: String, required: true },
7         modelo:     { type: String, required: true },
8         color:      { type: String, required: true },
9         matricula:  { type: String, required: true, uppercase: true }
10    },
11    ubicacionActual: {
12        lat: { type: Number, default: 39.5696 },
13        lng: { type: Number, default: 2.6502 }
14    },
15    estado: {
16        type: String,
17        enum: ['disponible', 'ocupado', 'desconectado'],
18        default: 'disponible'
19    },
20    valoracion:   { type: Number, default: 5.0, min: 0, max: 5 },
21    totalViajes:  { type: Number, default: 0 }
22 }, { timestamps: true });
```

Listing 3: Modelo Conductor (Mongoose)

```
1 const viajeSchema = new mongoose.Schema({
2     usuario:     { type: mongoose.Schema.Types.ObjectId, ref: 'Usuario',
3     required: true },
4     conductor:   { type: mongoose.Schema.Types.ObjectId, ref: 'Conductor',
5     required: true },
6     origen:      { type: String, required: true, trim: true },
7     destino:     { type: String, required: true, trim: true },
8     estado: {
9         type: String,
10        enum: ['asignado', 'en_camino', 'llegando', 'completado', 'cancelado'],
11        default: 'asignado'
12    }
```

```

10   },
11   precioEstimado:      { type: Number, required: true },
12   tiempoEstimadoMinutos: { type: Number, required: true },
13   horaInicio:          { type: Date, default: Date.now },
14   horaFinEstimada:      { type: Date, required: true },
15   progreso:            { type: Number, default: 0, min: 0, max:
100 },
16   valoracionUsuario:    { type: Number, min: 1, max: 5 },
17   comentario:           { type: String, trim: true }
18 }, { timestamps: true });

```

Listing 4: Modelo Viaje (Mongoose)

4.4. API REST - Endpoints

La API implementada actualmente incluye:

Método	Endpoint	Descripción
POST	/api/auth/register	Registro de nuevo usuario
POST	/api/auth/login	Autenticación de usuario
GET	/	Información de la API
POST	/api/viajes/solicitar	Solicitar un nuevo viaje
GET	/api/viajes/:id/estado	Estado y progreso del viaje
GET	/api/viajes/historial	Historial de viajes del usuario
POST	/api/viajes/:id/cancelar	Cancelar un viaje en curso
POST	/api/viajes/:id/valorar	Valorar un viaje completado

Cuadro 1: Endpoints de la API REST

4.5. Flujo de autenticación

1. El usuario completa el formulario de registro/login.
2. JavaScript valida los datos en el cliente.
3. Se envía petición POST al backend mediante Fetch API.
4. El backend valida los datos nuevamente.
5. Para registro: se encripta la contraseña con bcrypt y se guarda en MongoDB.
6. Para login: se compara la contraseña hasheada.
7. Si es exitoso, se devuelve información del usuario (sin contraseña).
8. El frontend almacena los datos en localStorage.
9. Se actualiza la interfaz mostrando información personalizada.

5. Desarrollo e implementación

5.1. Estado actual del proyecto (Versión UD1B)

El proyecto se encuentra en su segunda versión funcional. La versión UD1A estableció la base de autenticación y la interfaz inicial. La versión UD1B añade el sistema completo de solicitud y gestión de viajes. A continuación se detallan las características implementadas:

5.1.1. Sistema de autenticación completo

- **Registro de usuarios:**

- Formulario con validación en tiempo real.
- Campos: nombre, email, teléfono, contraseña, confirmación de contraseña.
- Mensajes de error específicos y traducidos.
- Redirección automática tras registro exitoso.

- **Login de usuarios:**

- Autenticación mediante email y contraseña.
- Validación de credenciales.
- Gestión de sesión con localStorage.
- Redirección a página principal.

- **Seguridad implementada:**

- Encriptación de contraseñas con bcrypt (10 rounds de salt).
- Validación exhaustiva en frontend y backend.
- Protección contra inyección de código.
- Manejo de errores robusto.

5.1.2. Interfaz de usuario

- **Página principal (index.html):**

- Hero section con formulario de solicitud de taxi.
- Campos de origen y destino.
- Botón de ubicación actual.
- Sugerencias de ubicaciones populares de Palma.

- Adaptación según estado de sesión del usuario.
- **Página de perfil (perfil.html):**
 - Información personal del usuario.
 - Sistema de insignias y estadísticas.
 - Historial de viajes (preparado para implementación).
 - Configuración de cuenta.
- **Página de conductores (conductores.html):**
 - Información sobre el proceso de verificación.
 - Requisitos para ser conductor.
 - Proceso de incorporación en 4 pasos.
 - Estadísticas del servicio (100 % verificados, 500+ conductores, 4.8).
 - Formulario CTA para solicitar ser conductor.
- **Página de ayuda (ayuda.html):**
 - Sección de preguntas frecuentes (FAQ) con 8 preguntas.
 - Información de contacto múltiple (teléfono, email, WhatsApp).
 - Horarios de atención.
 - Enlaces a redes sociales.
 - Barra de búsqueda (preparada para implementación).

5.1.3. Nuevas páginas del frontend (UD1B)

- **Página de viaje en curso (viaje-en-curso.html):**
 - Card del conductor con nombre, valoración y datos del vehículo.
 - Timeline visual animado con cuatro estados: Asignado, En camino, Llegando, Completado.
 - Barra de progreso actualizada cada 3 segundos.
 - Tiempo restante en minutos en cuenta regresiva.
 - Precio estimado visible en todo momento.
 - Botón de cancelación disponible hasta completarse el viaje.
 - Sistema de valoración (1–5 estrellas) al finalizar el viaje.
- **Página de historial de viajes (historial-viajes.html):**

- Listado de todos los viajes anteriores del usuario.
- Tabs para filtrar por estado: todos, completados, cancelados.
- Detalle de cada viaje: origen, destino, conductor, precio y valoración.

5.1.4. Sistema de viajes (UD1B)

■ Solicitud de viaje:

- El usuario introduce origen y destino desde la página principal.
- El sistema localiza automáticamente el conductor disponible.
- Se calculan precio estimado y tiempo de llegada.
- El conductor queda marcado como “ocupado” hasta finalizar el servicio.

■ Seguimiento en tiempo real:

- Polling cada 3 segundos al endpoint de estado.
- Progreso calculado en base a tiempos reales (`horaInicio` y `horaFinEstimada`).
- Transición automática de estados: `asignado` → `en_camino` → `llegando` → `completado`.
- Al completarse el viaje, el conductor vuelve a estar disponible.

■ Valoración:

- Sistema de 1 a 5 estrellas al finalizar el viaje.
- La valoración queda almacenada en el documento `Viaje` en MongoDB.

■ Script de datos de prueba (`seed-conductores.js`):

- Crea automáticamente 5 conductores de prueba en la base de datos.
- Cada conductor tiene datos completos: nombre, vehículo, ubicación en Palma y valoración inicial.

5.1.5. Despliegue en Vercel

- Configuración `vercel.json` que expone el backend de Node.js como función serverless.
- El frontend estático se sirve directamente desde Vercel CDN.
- Las rutas `/api/*` se redirigen al servidor Express y el resto al frontend.

5.1.6. Características de diseño

■ Diseño responsive:

- Adaptación automática a móviles, tablets y escritorio.
- Media queries para diferentes tamaños de pantalla.
- Menú de navegación adaptativo.

■ Sistema multiidioma:

- Soporte para español, inglés y catalán.
- Selector de idioma en navbar.
- Persistencia de preferencia en localStorage.
- Traducción completa de toda la interfaz.

■ Modales personalizados:

- Sistema de modales custom reemplazando alert() nativo.
- Diseño coherente con la identidad visual de la app.
- Animaciones suaves de entrada/salida.
- Botones personalizables (primarios y secundarios).

■ Paleta de colores corporativa:

- Color primario: #FDB913 (amarillo taxi característico).
- Gradientes: #FDB913 a #FFA500.
- Contraste alto para legibilidad.

5.2. Backend implementado

5.2.1. Servidor Express

```
1 require('dotenv').config();
2 const express = require('express');
3 const cors = require('cors');
4 const connectDB = require('./src/config/database');
5 const authRoutes = require('./src/routes/authRoutes');
6
7 const app = express();
8 const PORT = process.env.PORT || 3000;
9
10 // Conectar a la base de datos
```

```
11 connectDB();
12
13 // Middlewares
14 app.use(cors());
15 app.use(express.json());
16 app.use(express.urlencoded({ extended: true }));
17
18 // Rutas
19 app.use('/api/auth', authRoutes);
20
21 // Ruta de prueba
22 app.get('/', (req, res) => {
23   res.json({
24     mensaje: 'EasyTaxi Palma API',
25     version: '1.0.0',
26     estado: 'Activo'
27   });
28 });
29
30 // Inicio del servidor
31 app.listen(PORT, () => {
32   console.log('Servidor corriendo en puerto ${PORT}');
33 });
```

Listing 5: Configuración del servidor

5.2.2. Controlador de autenticación

El controlador `authController.js` implementa:

- Registro de usuarios con validación completa.
- Verificación de emails duplicados.
- Encriptación de contraseñas.
- Login con comparación de hashes.
- Manejo de errores específicos.

5.3. Base de datos

MongoDB se ejecuta mediante Docker Compose:

```
1 version: '3.8'
2
3 services:
4   mongodb:
```

```
5   image: mongo:7.0
6   container_name: easytaxi-mongodb
7   restart: always
8   ports:
9     - "27018:27017"
10  environment:
11    MONGO_INITDB_DATABASE: easytaxi
12  volumes:
13    - mongodb_data:/data/db
14
15 volumes:
16   mongodb_data:
```

Listing 6: docker-compose.yml

5.4. Sistema de ubicaciones

Actualmente implementado con base de datos local de JavaScript que incluye:

- Aeropuerto y transportes principales.
- Hoteles de 5 y 4 estrellas.
- Centros comerciales.
- Zonas turísticas (Catedral, Bellver, Paseo Marítimo).
- Playas principales.
- Hospitales y centros de salud.
- Puntos de interés cultural.

Total: más de 100 ubicaciones preprogramadas con información de nombre, dirección, icono y categoría.

6. Impacto y aspectos éticos

6.1. Accesibilidad

La aplicación se ha creado cumpliendo con buenas prácticas de accesibilidad web (WCAG 2.1):

- **Etiquetas ARIA:** Implementación de roles, states y properties para tecnologías asistivas.
- **Contraste de colores:** Ratio mínimo de 4.5:1 entre texto y fondo.
- **Diseño responsive:** Adaptación completa a diferentes tamaños de pantalla y dispositivos.
- **Navegación con teclado:** Todos los elementos interactivos son accesibles mediante Tab.
- **Texto legible:** Tipografía clara (Segoe UI) con tamaños adecuados.
- **Iconos con significado:** Uso de Phosphor Icons con labels descriptivos.
- **Formularios accesibles:** Labels asociados correctamente, mensajes de error claros.

De este modo, se asegura que cualquier individuo, incluso quienes presenten alguna limitación visual o motriz, pueda emplearla sin dificultades.

6.2. Usabilidad

EasyTaxi tiene una interfaz limpia e intuitiva siguiendo principios de UX/UI:

- **Simplicidad:** Procesos de registro y solicitud en pocos pasos.
- **Feedback inmediato:** Mensajes claros de éxito, error y validación.
- **Consistencia:** Patrones de diseño coherentes en todas las páginas.
- **Prevención de errores:** Validación en tiempo real de formularios.
- **Affordance:** Elementos claramente identificables como interactivos.
- **Multiidioma:** Adaptación al idioma preferido del usuario.

Se prioriza la experiencia del usuario por encima del aspecto visual complejo, buscando que personas de todas las edades puedan usar la aplicación, especialmente personas mayores que tradicionalmente tienen más dificultades con la tecnología.

6.3. Protección de datos (RGPD)

Se tratarán datos personales como nombre, email, ubicación y contraseñas. Para protegerlos se aplican medidas técnicas y organizativas:

6.3.1. Medidas técnicas implementadas

1. Cifrado de contraseñas:

- Uso de bcrypt con 10 rounds de salt.
- Las contraseñas nunca se almacenan en texto plano.
- Las contraseñas nunca se devuelven en respuestas de la API.

2. Validación exhaustiva:

- Validación en cliente (JavaScript).
- Validación en servidor (Mongoose + Express).
- Sanitización de inputs para prevenir inyección.

3. Conexión segura:

- HTTPS en producción (planificado).
- CORS correctamente configurado.

4. Gestión de sesiones:

- Actualmente: localStorage (desarrollo).
- Futuro: JWT con expiración y refresh tokens.

6.3.2. Medidas organizativas planificadas

- Política de privacidad accesible y clara.
- Consentimiento explícito para tratamiento de datos.
- Derecho al olvido (eliminación de cuenta).
- Portabilidad de datos.
- Minimización de datos: solo se solicitan datos necesarios.
- Transparencia en el uso de datos de ubicación.

Ningún dato se compartirá con terceros sin consentimiento explícito del usuario.

6.4. Licencias de software

Todo el software utilizado cumple con licencias de código abierto o gratuitas:

Tecnología	Licencia
Node.js	MIT License
Express.js	MIT License
MongoDB (Community Edition)	SSPL (Server Side Public License)
Mongoose	MIT License
bcryptjs	MIT License
Phosphor Icons	MIT License
Leaflet	BSD 2-Clause
OpenStreetMap	ODbL (Open Database License)

Cuadro 2: Licencias de software utilizadas

6.5. Diversidad e inclusión

EasyTaxi se centra principalmente en crear una interfaz muy sencilla y visible para incluir a las personas mayores que se quedan sin servicio por no poder usar tecnologías complejas o levantar la mano en la calle para detener un taxi.

- **Interfaz simplificada:** Procesos claros sin pasos innecesarios.
- **Textos grandes y legibles:** Evitando fuentes pequeñas o difíciles de leer.
- **Iconos descriptivos:** Complementando el texto con iconos claros.
- **Colores de alto contraste:** Facilitando la lectura.
- **Botones grandes:** Fáciles de pulsar en dispositivos táctiles.

Además, uno de los propósitos de la app es evitar que los taxistas hagan tantos kilómetros en vacío y puedan recoger a la persona más cercana para evitar gastos innecesarios y reducir el impacto medioambiental.

6.6. Impacto medioambiental

- **Reducción de kilómetros en vacío:** Asignación inteligente del taxi más cercano.
- **Optimización de rutas:** Menor consumo de combustible.
- **Digitalización del servicio:** Reducción de llamadas telefónicas y desplazamientos innecesarios.
- **Servidor eficiente:** Node.js tiene bajo consumo de recursos comparado con otras tecnologías.

7. Metodología de desarrollo

7.1. Metodología ágil con Trello

El desarrollo del proyecto sigue principios de metodología ágil utilizando Trello para la gestión de tareas:

- **Sprint planning:** Definición de objetivos para cada iteración (UD1A, UD1B, etc.).
- **User stories:** Definición de funcionalidades desde la perspectiva del usuario.
- **Tablero Kanban:** Columnas: Backlog, To Do, In Progress, Testing, Done.
- **Revisión continua:** Evaluación de cada funcionalidad implementada.

7.2. Control de versiones con Git

- Uso de Git para control de versiones local.
- Repositorio remoto en GitHub.
- Commits descriptivos siguiendo convenciones:
 - **feat:** nueva funcionalidad.
 - **fix:** corrección de errores.
 - **docs:** cambios en documentación.
 - **style:** cambios de formato/estilos.
 - **refactor:** refactorización de código.
- Branches para diferentes funcionalidades (planificado).

7.3. Testing y validación

- **Testing manual:** Pruebas exhaustivas de cada funcionalidad.
- **Testing en múltiples navegadores:** Chrome, Firefox, Safari, Edge.
- **Testing responsive:** Dispositivos móviles, tablets y escritorio.
- **Validación de código:** HTML Validator, CSS Validator.
- **Testing futuro:** Jest para backend, Cypress para e2e.

8. Próximos pasos y trabajo futuro

8.1. Fase UD1B - Sistema de viajes (Completada)

1. **Modelos Conductor y Viaje:** Implementados en MongoDB con Mongoose.
2. **API de viajes:** Endpoints para solicitar, consultar estado, listar historial, cancelar y valorar viajes.
3. **Seguimiento en tiempo real:** Timeline animado con progreso calculado dinámicamente.
4. **Script de seed:** `seed-conductores.js` para generar conductores de prueba.
5. **Despliegue Vercel:** Configuración serverless lista para producción.

8.2. Fase UD1C - Geolocalización y mapas

1. **Implementación de Leaflet:**
 - Integración de la librería de mapas.
 - Configuración de OpenStreetMap como proveedor.
 - Personalización de marcadores y estilos.
2. **Geolocalización del usuario:**
 - Uso de Geolocation API.
 - Solicitud de permisos.
 - Detección automática de ubicación.
3. **Visualización de taxis en el mapa:**
 - Marcadores de taxis disponibles en el mapa.
 - Animación del movimiento del taxi hacia el cliente.
 - Cálculo real de distancias mediante coordenadas.

8.3. Fase UD2 - Sistema de reservas

1. **Modelo de datos Reserva:**
 - Campos: usuario, conductor, origen, destino, fecha, estado, precio.
 - Relaciones con Usuario y Conductor.

2. Flujo de reserva:

- Selección de origen y destino.
- Cálculo de precio estimado.
- Asignación de taxi más cercano.
- Confirmación de reserva.
- Notificaciones en tiempo real.

3. Estados de reserva:

- Pendiente: esperando confirmación.
- Confirmado: taxi asignado.
- En camino: taxi dirigiéndose al punto de recogida.
- En curso: pasajero en el vehículo.
- Completado: viaje finalizado.
- Cancelado: reserva cancelada.

8.4. Fase UD3 - Funcionalidades avanzadas

1. Sistema de valoraciones:

- Valoración del servicio (1-5 estrellas).
- Comentarios opcionales.
- Historial de valoraciones por conductor.

2. Pagos integrados:

- Integración de pasarela de pago (Stripe).
- Pago con tarjeta en la app.
- Historial de pagos.
- Facturación automática.

3. Notificaciones push:

- Confirmación de reserva.
- Taxi en camino.
- Llegada del taxi.
- Finalización del servicio.

4. Chat en tiempo real:

- Conversación entre usuario y conductor.
- Socket.io para comunicación bidireccional.

8.5. Mejoras de seguridad

- **Implementación completa de JWT** (sustituir el middleware temporal de email por header).
- Autenticación de dos factores (2FA).
- Rate limiting en la API.
- Logs de auditoría.
- Certificado SSL en producción (gestionado por Vercel).

8.6. Optimizaciones

- Caché de ubicaciones populares.
- Lazy loading de imágenes.
- Minificación de CSS y JavaScript.
- Service Workers para PWA.
- Optimización de consultas a MongoDB.

8.7. Expansión futura

- Aplicación móvil nativa (React Native).
- Panel de administración para gestión de conductores.
- Sistema de bonos y promociones.
- Integración con servicios de transporte público.
- Expansión a otras ciudades de Mallorca.

9. Conclusiones

9.1. Logros alcanzados

El desarrollo de EasyTaxi Palma en sus fases UD1A y UD1B ha permitido alcanzar los siguientes logros:

1. **Sistema funcional de autenticación:** Implementación completa de registro y login con las mejores prácticas de seguridad.
2. **Sistema completo de viajes:** Solicitud, seguimiento en tiempo real con timeline animado, cancelación y valoración de viajes.
3. **API REST ampliada:** Diez endpoints funcionales cubriendo autenticación y gestión de viajes.
4. **Modelos de datos relacionados:** Esquemas Mongoose para Usuario, Conductor y Viaje con relaciones entre documentos.
5. **Arquitectura serverless:** Configuración de despliegue en Vercel con frontend estático y backend como función serverless.
6. **Interfaz profesional:** Diseño moderno, responsive y multiidioma con páginas adicionales de viaje en curso e historial.
7. **Cumplimiento de requisitos éticos:** Implementación de medidas de accesibilidad, usabilidad y protección de datos.

9.2. Competencias adquiridas

Durante el desarrollo del proyecto he desarrollado y consolidado competencias clave:

- **Desarrollo full-stack:** Capacidad de trabajar tanto en frontend como en backend.
- **Diseño de APIs REST:** Arquitectura y implementación de endpoints eficientes.
- **Seguridad web:** Implementación de medidas de protección de datos y contraseñas.
- **Bases de datos NoSQL:** Diseño de esquemas y consultas con MongoDB.
- **UX/UI Design:** Creación de interfaces intuitivas y accesibles.
- **Control de versiones:** Uso profesional de Git y GitHub.

- **Documentación técnica:** Creación de documentación clara y completa.
- **Metodología ágil:** Gestión de proyecto mediante iteraciones y sprints.

9.3. Retos superados

- **Validación completa:** Implementar validación robusta tanto en cliente como en servidor.
- **Sistema multiidioma:** Diseñar una solución escalable para internacionalización.
- **Modales personalizados:** Reemplazar elementos nativos del navegador manteniendo funcionalidad.
- **Responsive design:** Adaptar la interfaz a múltiples tamaños de pantalla sin perder usabilidad.
- **Gestión de estado:** Sincronizar el estado de sesión entre diferentes páginas.
- **Seguimiento en tiempo real sin WebSockets:** Implementar polling eficiente para actualizar el estado del viaje cada 3 segundos.
- **Gestión del ciclo de vida del conductor:** Liberar automáticamente al conductor al completarse el viaje.
- **Configuración serverless:** Adaptar el servidor Express para funcionar tanto en local como como función serverless en Vercel.

9.4. Reflexión personal

Este proyecto representa mucho más que un trabajo académico para mí. Es la materialización de una necesidad real que identifiqué durante mi experiencia laboral como taxista. He podido aplicar conocimientos técnicos para aportar una solución a un sector que necesita urgentemente modernizarse.

El desarrollo ha sido constantemente desafiante, obligándome a investigar, aprender nuevas tecnologías y resolver problemas complejos. Cada funcionalidad implementada me ha aportado aprendizaje valioso que sé que aplicaré en mi futura carrera profesional.

Estoy satisfecho con los resultados obtenidos hasta ahora, y motivado para continuar desarrollando las próximas fases del proyecto. EasyTaxi Palma tiene potencial para convertirse en una aplicación real que pueda ayudar tanto a taxistas como a usuarios de Palma de Mallorca.

9.5. Viabilidad del proyecto

EasyTaxi Palma es un proyecto técnicamente viable y necesario:

- **Necesidad real del mercado:** Existe demanda de soluciones digitales en el sector del taxi.
- **Tecnologías maduras:** Stack tecnológico probado y estable.
- **Escalabilidad:** Arquitectura preparada para crecer.
- **Coste reducido:** Uso de tecnologías open source minimiza costes.
- **Base funcional:** El proyecto ya tiene una base sólida sobre la que construir.

Con las próximas fases de desarrollo completadas, el proyecto podría evolucionar hacia un MVP (Minimum Viable Product) que podría presentarse a asociaciones de taxistas y realizar pruebas piloto reales.

10. Bibliografía y referencias

10.1. Documentación oficial

1. Node.js Documentation. (2026). *Node.js Official Documentation*. Recuperado de <https://nodejs.org/docs/>
2. Express.js Guide. (2026). *Express.js Official Documentation*. Recuperado de <https://expressjs.com/>
3. MongoDB Manual. (2026). *MongoDB Official Documentation*. Recuperado de <https://docs.mongodb.com/>
4. Mongoose Documentation. (2026). *Mongoose ODM*. Recuperado de <https://mongoosejs.com/docs/>
5. MDN Web Docs. (2026). *JavaScript — MDN*. Mozilla Developer Network. Recuperado de <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
6. Leaflet Documentation. (2026). *Leaflet - a JavaScript library for interactive maps*. Recuperado de <https://leafletjs.com/>

10.2. Libros y recursos educativos

7. Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.
8. Young, A., Meck, B., & Cantelon, M. (2017). *Node.js in Action* (2nd ed.). Manning Publications.
9. Hahn, E. (2014). *Express in Action: Writing, building, and testing Node.js applications*. Manning Publications.
10. Banker, K. (2016). *MongoDB in Action* (2nd ed.). Manning Publications.

10.3. Artículos y guías web

11. OWASP Foundation. (2021). *OWASP Top Ten Web Application Security Risks*. Recuperado de <https://owasp.org/www-project-top-ten/>
12. W3C. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. Recuperado de <https://www.w3.org/WAI/WCAG21/quickref/>
13. Google Developers. (2026). *Web Fundamentals*. Recuperado de <https://developers.google.com/web/fundamentals>

14. Nielsen Norman Group. (2020). *Usability Heuristics for User Interface Design*. Recuperado de <https://www.nngroup.com/articles/ten-usability-heuristics/>

10.4. Herramientas y librerías

15. Phosphor Icons. (2026). *A flexible icon family for interfaces, diagrams, presentations*. Recuperado de <https://phosphoricons.com/>
16. Docker Documentation. (2026). *Docker Documentation*. Recuperado de <https://docs.docker.com/>
17. Git Documentation. (2026). *Git Reference Manual*. Recuperado de <https://git-scm.com/docs>

10.5. Legislación y normativa

18. Parlamento Europeo. (2016). *Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales (RGPD)*.
19. Agencia Española de Protección de Datos. (2018). *Guía para el cumplimiento del RGPD*. Recuperado de <https://www.aepd.es/>
20. Gobierno de las Islas Baleares. (2023). *Normativa del servicio de taxi en Palma de Mallorca*. Consejería de Movilidad y Vivienda.

10.6. Recursos de consulta durante el desarrollo

21. Stack Overflow. (2026). *Stack Overflow - Where Developers Learn, Share, & Build Careers*. Recuperado de <https://stackoverflow.com/>
22. GitHub. (2026). *Open source projects and repositories*. Recuperado de <https://github.com/>
23. CSS-Tricks. (2026). *CSS-Tricks - Tips, Tricks, and Techniques on using CSS*. Recuperado de <https://css-tricks.com/>

Anexos

Anexo A: Capturas de pantalla

Nota: En la versión final se incluirían capturas de pantalla de:

- Página principal (index.html)
- Formulario de registro (register.html)
- Formulario de login (login.html)
- Página de perfil de usuario (perfil.html)
- Página de conductores (conductores.html)
- Página de ayuda (ayuda.html)
- Vista responsive en diferentes dispositivos
- Modales personalizados
- Selector de idiomas

Anexo B: Diagrama de base de datos

Nota: En la versión final se incluiría un diagrama E-R (Entidad-Relación) mostrando:

- Entidad Usuario (actual)
- Entidad Conductor (planificada)
- Entidad Reserva (planificada)
- Relaciones entre entidades

Anexo C: Flujos de usuario

Nota: En la versión final se incluirían diagramas de flujo de:

- Proceso de registro
- Proceso de login
- Flujo de solicitud de taxi (planificado)
- Flujo de reserva (planificado)

Anexo D: Código fuente relevante

El código completo está disponible en el repositorio GitHub del proyecto.

Extractos de código más relevantes incluidos en el documento:

- Modelo de Usuario (Mongoose)
- Modelo de Conductor (Mongoose)
- Modelo de Viaje (Mongoose)
- Configuración del servidor Express
- Docker Compose para MongoDB
- Configuración de despliegue Vercel (vercel.json)

Declaración de autoría

Yo, **Sebastià Romaguera Camps**, declaro que este Trabajo de Fin de Grado titulado “*EasyTaxi Palma: Aplicación web para la reserva y petición de taxis en Palma de Mallorca*” ha sido realizado íntegramente por mí, bajo la orientación y supervisión de los profesores del ciclo formativo de Desarrollo de Aplicaciones Web.

Todo el trabajo ha sido escrito por mí, incluyendo:

- Análisis de requisitos
- Diseño de la arquitectura del sistema
- Implementación del código (frontend y backend)
- Documentación técnica
- Este documento

Salvo en los casos específicos que se indican a continuación, el trabajo es completamente original:

- He utilizado inteligencia artificial (GitHub Copilot y ChatGPT) como herramienta de consulta para:
 - Decisiones sobre qué tecnologías usar y por qué.
 - Resolución de dudas técnicas específicas.
 - Sugerencias de buenas prácticas de programación.
- He consultado documentación oficial de las tecnologías empleadas.
- He revisado ejemplos de código en Stack Overflow y GitHub para resolver problemas específicos.

Todas las fuentes consultadas están debidamente citadas en la sección de bibliografía de este documento.

Es un proyecto que tengo ganas de crear ya que he vivido de primera mano lo que es el sector del taxi y quiero mejorarlo mediante la tecnología. La motivación personal y la experiencia real trabajando como taxista son el motor principal de este proyecto.

Declaro que este trabajo no ha sido presentado anteriormente para la obtención de ningún título académico y que no contiene material copiado de otras fuentes sin la debida atribución.

Sebastià Romaguera Camps

Palma de Mallorca, Febrero de 2026